Automatizaciones con LangChain

Eventbrite · BullMQ · Express · Mongoose · JavaScript

Cadenas inteligentes con router de IA para notificaciones y marketing de eventos

Arquitectura general

El sistema conecta la API de Eventbrite con un conjunto de cadenas LangChain especializadas. Un router de IA decide automaticamente cual cadena ejecutar en funcion del contexto: datos del evento, historial de asistentes en MongoDB y la accion que disparo el webhook.

Stack tecnologico

- Express.js — servidor + webhook receiver
- BullMQ + Redis — colas de jobs y cron
- Mongoose + MongoDB — historial de usuarios
- LangChain JS — cadenas y router de IA
- Eventbrite API v3 — fuente de datos y descuentos

Canales de salida

- Email transaccional (Resend / SendGrid)
- Web push notifications (OneSignal)
- SMS / WhatsApp (Twilio)
- Post en Facebook / Instagram
- Descuentos automaticos en Eventbrite

Router de IA — el cerebro del sistema

En lugar de ejecutar cadenas por condiciones fijas en codigo, un LLM actua como router: recibe la accion del webhook y el contexto, y decide que cadena es la mas apropiada. Esto permite anadir nuevas cadenas sin modificar la logica de enrutamiento.

Endpoint Express

```
// routes/automations.js
const express = require('express');
const router  = express.Router();
const { routerChain } = require('../chains/router');

router.post('/webhook/eventbrite', async (req, res) => {
  const { config, api_url } = req.body;
  const result = await routerChain.invoke({
    action:  config.action,
    api_url,
```

```
    eventId: config.event_id
  });
  res.json({ ok: true, chain: result.selectedChain });
});

module.exports = router;
```

Cadena router (LangChain)

```
// chains/router.js
const { ChatOpenAI }    = require('@langchain/openai');
const { PromptTemplate } = require('@langchain/core/prompts');
const { StringOutputParser } = require('@langchain/core/output_parsers');
const chains = require('./index');

const llm = new ChatOpenAI({ model: 'gpt-4o-mini', temperature: 0 });

const routerPrompt = PromptTemplate.fromTemplate(`
Eres el router de un sistema de automatizacion de eventos.
Accion de Eventbrite: {action}
Decide que cadena ejecutar:
- low_sales_discount: ventas bajas, pocos tickets vendidos
- age_facebook_campaign: audiencia mayor de 30, campaña Facebook
- checkin_welcome: asistente hace check-in en el evento
- post_event_survey: evento terminado hace <2h, encuesta
- last_minute_push: faltan <48h y hay >20% entradas libres
- vip_upsell: asistente tiene historial de >3 eventos
Responde SOLO con el nombre de la cadena.`);

const routerChain = routerPrompt.pipe(llm).pipe(new StringOutputParser())
  .pipe(async (chainName) => {
    const chain = chains[chainName.trim()];
    if (!chain) throw new Error(`Cadena desconocida: ${chainName}`);
    return chain;
  });

module.exports = { routerChain };
```

Modelos Mongoose

Los datos de usuarios e historico de asistencias se almacenan en MongoDB. Estos modelos son la fuente que alimenta las decisiones de las cadenas: segmentacion por edad, deteccion de VIPs, anti-spam de notificaciones, etc.

```
// models/User.js
const mongoose = require('mongoose');

const UserSchema = new mongoose.Schema({
  email:            { type: String, required: true, unique: true },
  name:             String,
  age:              Number,
  city:             String,
  phone:            String,
```

```
  pushSubscription: String,   // OneSignal external_id
  interestedEvents: [String], // event IDs de Eventbrite
  purchasedEvents:  [String],
  visitedEvents:    [String],
  waitlistEvents:   [String],
  notifiedDiscount: [String], // anti-spam
  preferences:      [String],
}, { timestamps: true });

// models/Attendee.js - historico de asistencias
const AttendeeSchema = new mongoose.Schema({
  email:       { type: String, required: true },
  organizerId: String,
  eventId:     String,
  eventType:   String,
  age:         Number,
  city:        String,
  ticketPrice: Number,
  checkedIn:   Boolean,
  attended:    Boolean,
  interests:   [String],
  pastEvents:  [String],
  surveyAnswered: Boolean,
}, { timestamps: true });
```

Setup BullMQ + Cron

Cada automatizacion se ejecuta como un job en BullMQ. Los jobs periodicos (comprobacion de ventas) se registran al arrancar la app. Los webhooks de Eventbrite encolan jobs on-demand.

```
// queues/index.js
const { Queue, Worker } = require('bullmq');
const Redis = require('ioredis');

const connection = new Redis(process.env.REDIS_URL);

const automationQueue = new Queue('automations', { connection });

// Registrar jobs repetitivos al arrancar
async function registerCronJobs(events) {
  for (const event of events) {
    await automationQueue.add(
      'check-sales',
      { eventId: event.id, threshold: 0.4 },
      { repeat: { every: 60 * 60 * 1000 }, jobId: `sales-${event.id}` }
    );
  }
}

module.exports = { automationQueue, registerCronJobs };
```

Cadenas disponibles

Cada cadena sigue el mismo patron: consulta MongoDB para contexto, llama a LangChain para generar contenido inteligente, actua sobre el canal de salida adecuado.

Cadena 1: Descuento por ventas bajas

Trigger: Cron job cada hora o webhook order.placed

Steps de la cadena

1. Obtener ticket_classes del evento via Eventbrite API
2. Calcular porcentaje vendido (quantity_sold / quantity_total)
3. Si < umbral (ej: 40%), consultar usuarios interesados en MongoDB
4. Generar email urgente personalizado con LangChain
5. Crear codigo de descuento en Eventbrite (POST /discounts/)
6. Enviar emails en batch con Resend
7. Marcar usuarios como notificados (anti-spam en MongoDB)

Consulta Mongoose

```
// Obtener usuarios interesados en el evento
const users = await User.find({
  interestedEvents: eventId,
  notifiedDiscount: { $ne: eventId }  // no notificados aun
}).select('email name preferences');
```

Cadena LangChain (JS)

```
const { ChatOpenAI }      = require('@langchain/openai');
const { PromptTemplate }   = require('@langchain/core/prompts');
const { RunnableSequence } = require('@langchain/core/runnables');
const Resend = require('resend').Resend;

const llm    = new ChatOpenAI({ model: 'gpt-4o-mini' });
const resend = new Resend(process.env.RESEND_API_KEY);

const lowSalesChain = RunnableSequence.from([
  // Step 1: analizar el contexto del evento
  PromptTemplate.fromTemplate(`
    El evento '{eventName}' tiene solo {pct}% de entradas vendidas.
    Faltan {days} dias. Crea un email persuasivo con:
    - Asunto urgente (max 60 chars)
    - Cuerpo corto (max 80 words)
    - Incluye el codigo descuento: {code}
    - Tono: amigable pero urgente
    Responde en JSON: { subject, body }`),
  llm,
  // Step 2: parsear respuesta
  async (res) => JSON.parse(res.content),
  // Step 3: enviar emails en batch
  async ({ subject, body }) => {
    await Promise.allSettled(
      users.map(u => resend.emails.send({
```

```
        from: 'eventos@tudominio.com',
        to:   u.email,
        subject,
        html: `<p>Hola ${u.name},</p><p>${body}</p>`
      }))
    );
    // Marcar como notificados
    await User.updateMany(
      { _id: { $in: users.map(u => u._id) } },
      { $push: { notifiedDiscount: eventId } }
    );
  }
]);
```

Accion final

Email masivo con codigo descuento auto-generado en Eventbrite. Los usuarios marcados no vuelven a recibir el mismo descuento.

Cadena 2: Campana Facebook por perfil de edad

Trigger: Cron previo al evento (7 dias antes) o trigger manual

Steps de la cadena

1. Calcular media de edad de asistentes previos (Mongoose aggregate)
2. Si edad media >= 30 → canal Facebook; si < 30 → canal Instagram
3. Obtener datos completos del evento via Eventbrite API (con expand=venue)
4. Generar copy del post con LangChain segun audiencia y evento
5. Publicar post via Facebook Graph API con foto del evento
6. Crear codigo descuento exclusivo en Eventbrite
7. Enviar email a usuarios registrados con link al post + codigo como guino

Consulta Mongoose

```
// Calcular media de edad de asistentes previos
const stats = await Attendee.aggregate([
  { $match: { organizerId, attended: true } },
  { $group: {
      _id: null,
      avgAge:   { $avg: '$age' },
      topCity:  { $first: '$city' },
      interests: { $push: '$interests' }
  }}
]);
const { avgAge, topCity } = stats[0];
```

Cadena LangChain (JS)

```
const fbChain = RunnableSequence.from([
  // Step 1: decidir canal segun edad
  async (input) => {
    if (input.avgAge < 30) return { ...input, channel: 'instagram' };
    return { ...input, channel: 'facebook' };
```

```
  },
  // Step 2: obtener datos del evento desde Eventbrite
  async (input) => {
    const event = await fetch(
      `https://www.eventbriteapi.com/v3/events/${input.eventId}/?expand=venue`,
      { headers: { Authorization: `Bearer ${process.env.EB_TOKEN}` } }
    ).then(r => r.json());
    return { ...input, event };
  },
  // Step 3: generar copy del post con LangChain
  PromptTemplate.fromTemplate(`
    Crea un post de {channel} para el evento:
    Nombre: {event.name.text}
    Fecha: {event.start.local}
    Lugar: {event.venue.name}, {event.venue.address.city}
    Audiencia media: {avgAge} anos, ciudad: {topCity}
    Incluye 3 hashtags relevantes. Max 280 chars.`,),
  llm,
  // Step 4: crear el post via API de Facebook
  async (res) => {
    const postText = res.content;
    await fetch(
      `https://graph.facebook.com/v19.0/${process.env.FB_PAGE_ID}/feed`,
      { method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          message: postText,
          access_token: process.env.FB_ACCESS_TOKEN
        })
      }
    );
    // Step 5: email a usuarios con link al post + codigo descuento
    const code = `FB20_${Date.now()}`;
    await createEventbriteDiscount(eventId, code, 20);
    await notifyUsersEmail(users, {
      subject: 'Tenemos algo para ti como guino  ;)',
      code,
      fbLink: `https://facebook.com/${process.env.FB_PAGE_ID}`
    });
  }
]);
```

Accion final

Post en Facebook/Instagram generado por IA + email a usuarios con codigo descuento exclusivo. El guino genera engagement y trackea la conversion del post.

Cadena 3: Bienvenida personalizada en check-in

Trigger: Webhook attendee.checked_in (tiempo real)

Steps de la cadena

1. Recibir webhook attendee.checked_in de Eventbrite

2. Buscar historial del asistente en MongoDB (eventos previos, preferencias)

3. Determinar si es VIP (>= 3 eventos con el organizador)

4. Generar mensaje de bienvenida personalizado con LangChain

5. Enviar push notification via OneSignal en tiempo real

6. Si es VIP, anadir badge VIP en la BD del organizador

Consulta Mongoose

```
// Historial del asistente: cuantos eventos ha asistido
const profile = await Attendee.findOne({ email: attendeeEmail })
  .select('name pastEvents preferences city');

const isVIP = profile?.pastEvents?.length >= 3;
```

Cadena LangChain (JS)

```
const checkinChain = RunnableSequence.from([
  // Step 1: enriquecer con historial
  async (input) => ({
    ...input,
    isVIP:     input.profile.pastEvents.length >= 3,
    firstName: input.profile.name.split(' ')[0]
  }),
  // Step 2: generar mensaje personalizado
  PromptTemplate.fromTemplate(`
    El asistente {firstName} acaba de hacer check-in.
    Ha asistido a {pastCount} eventos anteriores. VIP: {isVIP}.
    Crea un mensaje de bienvenida push notification (max 100 chars).
    Si es VIP, menciónalo sutilmente. Tono calido.`),
  llm,
  // Step 3: enviar push via OneSignal
  async (res) => {
    await fetch('https://onesignal.com/api/v1/notifications', {
      method: 'POST',
      headers: {
        'Authorization': `Basic ${process.env.ONESIGNAL_API_KEY}`,
        'Content-Type': 'application/json'
      },
      body: JSON.stringify({
        app_id: process.env.ONESIGNAL_APP_ID,
        include_external_user_ids: [attendeeEmail],
        contents: { en: res.content }
      })
    });
  }
]);
```

Accion final

Push notification personalizada en los segundos posteriores al check-in. Los VIPs reciben un trato diferenciado que fideliza.

Cadena 4: Encuesta post-evento generada por IA

Trigger: Webhook event.updated (cuando termina) o cron 2h despues del fin

Steps de la cadena

1. Detectar que el evento ha terminado (end time < now)
2. Obtener asistentes que hicieron check-in desde MongoDB
3. Generar preguntas de encuesta con LangChain adaptadas al tipo de evento
4. Crear encuesta en MongoDB con ID unico por usuario
5. Enviar email con enlace personalizado a la encuesta
6. Agregar respuestas y actualizar perfil de preferencias del asistente

Consulta Mongoose

```
// Asistentes que hicieron check-in en el evento
const checkedIn = await Attendee.find({
  eventId,
  checkedIn: true,
  surveyAnswered: { $ne: true }
}).select('email name rating');

// Media de valoracion si ya hay respuestas previas
const prevRatings = await Survey.aggregate([
  { $match: { organizerId } },
  { $group: { _id: null, avg: { $avg: '$score' } } }
]);
```

Cadena LangChain (JS)

```
const surveyChain = RunnableSequence.from([
  // Step 1: generar pregunta personalizada segun el evento
  PromptTemplate.fromTemplate(`
    Crea una encuesta post-evento de 3 preguntas para:
    Evento: {eventName}. Tipo: {eventType}.
    Incluye: satisfaccion general, aspecto favorito, mejora sugerida.
    Formato JSON: [{ question, type: 'rating'|'text' }]`),
  llm,
  async (res) => JSON.parse(res.content),
  // Step 2: crear survey en tu BD
  async (questions) => {
    const survey = await Survey.create({ eventId, questions, createdAt: new Date()
});
    return { surveyId: survey._id, questions };
  },
  // Step 3: enviar email con enlace personalizado
  async ({ surveyId }) => {
    const baseUrl = process.env.APP_URL;
    await Promise.allSettled(
      checkedIn.map(u =>
        resend.emails.send({
          to:      u.email,
          subject: 'Cuéntanos qué tal el evento  (2 min)',
          html:    `<p>Hola ${u.name}!<br>
                   <a href='${baseUrl}/survey/${surveyId}?u=${u._id}'>
                   Responder encuesta</a></p>`
        })
      )
    );
  }
]);
```

Accion final

Encuesta personalizada por tipo de evento, generada automaticamente. Las respuestas enriquecen el perfil de preferencias para futuras segmentaciones.

Cadena 5: Ultimo aviso: entradas agotandose

Trigger: Cron 48h antes del evento si quedan entradas libres

Steps de la cadena

1. Calcular horas restantes y entradas disponibles (Eventbrite API)
2. Consultar usuarios en lista de espera o que visitaron sin comprar (MongoDB)
3. Generar mensaje de maxima urgencia con datos reales (LangChain)
4. Enviar push notification via OneSignal a suscriptores
5. Enviar SMS via Twilio a usuarios con telefono registrado
6. Actualizar estado de notificacion en MongoDB

Consulta Mongoose

```
// Usuarios en lista de espera o que visitaron la pagina
const waitlist = await User.find({
  $or: [
    { waitlistEvents: eventId },
    { visitedEvents: eventId, purchasedEvents: { $ne: eventId } }
  ]
}).select('email name pushSubscription');
```

Cadena LangChain (JS)

```
const lastMinuteChain = RunnableSequence.from([
  // Step 1: calcular datos de urgencia
  async (input) => {
    const hoursLeft   = Math.round(
      (new Date(input.eventStart) - Date.now()) / 3600000
    );
    const ticketsLeft = input.totalCap - input.sold;
    return { ...input, hoursLeft, ticketsLeft };
  },
  // Step 2: generar mensaje de urgencia
  PromptTemplate.fromTemplate(`
    Quedan {hoursLeft} horas para '{eventName}'.
    Solo {ticketsLeft} entradas disponibles.
    Crea una push notification de MAXIMA urgencia (max 90 chars).
    Incluye numeros concretos. Sin signos de exclamacion.`),
  llm,
  // Step 3: enviar push + SMS a los interesados
  async (res) => {
    const msg = res.content;
    // Push via OneSignal
    await sendPushBatch(waitlist.filter(u => u.pushSubscription), msg);
    // SMS via Twilio para quienes tienen telefono
    const withPhone = waitlist.filter(u => u.phone);
    await Promise.allSettled(
      withPhone.map(u => twilioClient.messages.create({
        to:   u.phone,
```

```
        from: process.env.TWILIO_NUMBER,
        body: msg
      }))
    );
  }
]);
```

Accion final

Multicanal (push + SMS) con mensaje generado en base a datos reales de disponibilidad. Sin signos de exclamacion — la urgencia la dan los numeros.

Cadena 6: Upsell VIP personalizado

Trigger: Cuando se publica un nuevo evento (webhook event.published)

Steps de la cadena

1. Detectar publicacion de nuevo evento via webhook event.published

2. Identificar asistentes VIP del organizador (>= 3 eventos, top gasto)

3. Analizar patron de preferencias de cada VIP con LangChain

4. Generar oferta exclusiva personalizada (headline, descuento, CTA)

5. Crear codigo descuento VIP unico en Eventbrite por persona

6. Enviar email con diseno especial VIP y oferta personalizada

Consulta Mongoose

```
// Detectar asistentes VIP: >3 eventos con el mismo organizador
const vipAttendees = await Attendee.aggregate([
  { $match: { organizerId, attended: true } },
  { $group: {
      _id: '$email',
      count:       { $sum: 1 },
      totalSpent:  { $sum: '$ticketPrice' },
      lastEvent:   { $max: '$eventDate' },
      preferences: { $push: '$eventType' }
  }},
  { $match: { count: { $gte: 3 } } },
  { $sort:  { totalSpent: -1 } }
]);
```

Cadena LangChain (JS)

```
const vipUpsellChain = RunnableSequence.from([
  // Step 1: analizar patron de preferencias del VIP
  PromptTemplate.fromTemplate(`
    Cliente VIP con {count} eventos asistidos.
    Gasto total: {totalSpent}EUR.
    Tipos de eventos: {preferences}.
    Nuevo evento disponible: {eventName} - {eventType}.
    Genera una oferta exclusiva personalizada (JSON):
    { headline, body, discountPct, callToAction }`),
  llm,
  async (res) => JSON.parse(res.content),
```

```
  // Step 2: crear descuento VIP en Eventbrite
  async (offer) => {
    const code = `VIP${offer.discountPct}_${vip._id.slice(-4).toUpperCase()}`;
    await createEventbriteDiscount(eventId, code, offer.discountPct);
    return { ...offer, code };
  },
  // Step 3: email VIP con diseño especial
  async ({ headline, body, code, callToAction }) => {
    await resend.emails.send({
      to:      vip._id,  // email del VIP
      subject: `Solo para ti: ${headline}`,
      html:    vipEmailTemplate({ headline, body, code, callToAction })
    });
  }
]);
```

Accion final

Oferta 1:1 generada por IA para cada VIP. El descuento es unico por persona, lo que aumenta la conversion y la percepcion de exclusividad.

Tabla de referencia rapida

Cadena	Trigger	Datos MongoDB	Salida
Low sales discount	Cron / order.placed	interestedEvents, notifiedDiscount	Email masivo + descuento Eventbrite
Age Facebook campaign	Cron 7 dias antes	avgAge, topCity, preferences	FB/IG post + email con codigo
Check-in welcome	attendee.checked_in	pastEvents, checkedIn	Push notification OneSignal
Post-event survey	Cron 2h post-evento	checkedIn, surveyAnswered	Email con encuesta personalizada
Last-minute push	Cron 48h antes	waitlistEvents, visitedEvents	Push + SMS multicanal
VIP upsell	event.published	totalSpent, count, preferences	Email VIP + descuento exclusivo

Variables de entorno (.env)

```
# Eventbrite
EB_TOKEN=your_eventbrite_oauth_token

# LangChain / OpenAI
OPENAI_API_KEY=sk-...
```

```
# Redis (BullMQ)
REDIS_URL=redis://localhost:6379

# MongoDB
MONGODB_URI=mongodb://localhost:27017/events-automation

# Email
RESEND_API_KEY=re_...

# Push notifications
ONESIGNAL_APP_ID=...
ONESIGNAL_API_KEY=...

# SMS
TWILIO_ACCOUNT_SID=AC...
TWILIO_AUTH_TOKEN=...
TWILIO_NUMBER=+34...

# Facebook
FB_PAGE_ID=...
FB_ACCESS_TOKEN=...

# App
APP_URL=https://tudominio.com
```

Instalacion

```
npm install express bullmq ioredis mongoose
npm install @langchain/openai @langchain/core langchain
npm install resend twilio
```